

Project Reflection — Epic Brawlers

Anh Huynh

This project helped me better understand how object-oriented design concepts come together in a complete application rather than existing as isolated ideas. Through building the game, I discovered how important class responsibility, separation of concerns, and incremental testing are when working on a larger system. Features like the shop, save/load system, and boss mechanics forced me to think carefully about how data flows between objects and how changes in one area can affect the entire program. One of the easier parts of the project was implementing basic gameplay logic such as attacking, dealing damage, and earning gold. Once the core classes were set up, adding new weapons or bosses followed a predictable pattern. Seeing the game visually respond through the GUI made these parts especially satisfying and reinforced my understanding of event-driven programming. It was rewarding to see how small logic changes immediately reflected in the user interface. The most challenging part of the project was debugging late-game issues and scaling problems. A major obstacle occurred when the player began instantly dying around Stage 44 due to overflow and HP scaling errors after switching health values to long. Additionally, player damage unexpectedly dropping to 1 after surpassing the integer threshold made it confusing at first to understand why combat behavior changed so drastically. Tracking this down required careful testing, reviewing formulas, and refactoring both combat logic and save/load behavior. I overcame these challenges by breaking the problem into smaller pieces, adding logs, and testing one change at a time instead of guessing. I worked on this project alone, which required strong time management and self-discipline (taking 3 upper division classes). Overall, this project significantly improved my confidence in programming. I now feel more comfortable debugging complex issues, designing systems that scale, and iterating on a project over time. For future students, my advice is to test early and often, keep changes small, and not be discouraged by bugs, as debugging is where the most learning happens.